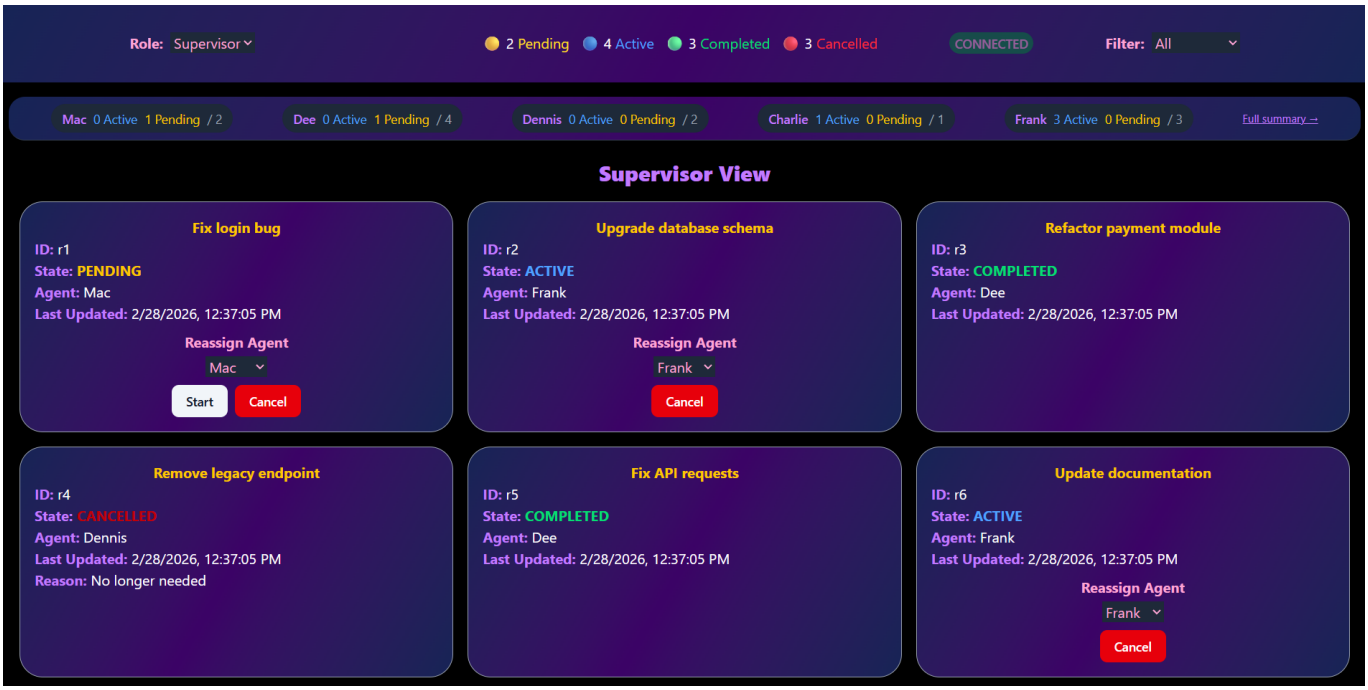
Operations Dashboard



A real-time operations dashboard built with React 19, TypeScript, and Tailwind CSS v4. It simulates a live request management system where supervisors can monitor, reassign, and manage requests across multiple agents, while agents see and control only their own assigned work.

Running Locally

```
git clone https://github.com/BinitAcharya7/Multi-Role-Dashboard && cd depth-nepal-project
npm install
npm run dev
```

Open http://localhost:5173 in your browser.

Tech Stack

- **React 19** + **TypeScript** — strict typing with discriminated union actions
- **Vite 7** — fast dev server and build
- **Tailwind CSS v4** — styling with responsive breakpoints
- **shadcn/ui** — reusable `Button` component via Radix UI primitives

Architecture Decisions

State Management — `useReducer` + Context

All app state lives in a single `useReducer` managed by `AppContext`. This was chosen over external libraries (Redux, Zustand) because I am more familiar with this way, and the state is not that complex.

Each reducer handler is extracted into its own function (`handleStartRequest`, `handleCancelRequest`, etc.) to keep the switch statement clean.

Simulated Event Stream — `useRequestStream`

The `useRequestStream` hook simulates a WebSocket connection:

- Every **5 seconds**, one of three random events fires: a new request is created, an existing request advances state, or an agent is assigned to an unassigned request
- Every **30 seconds**, the stream "disconnects" and reconnects after 5 seconds, cycling indefinitely
- A `ref` holds the latest `state.requests` so the effect doesn't restart on every state change. This was very important to prevent the disconnect timer from being cleared before it fires
- Returns a `CONNECTED` | `RECONNECTING` status displayed as a live indicator in the header

Request State Machine

Requests follow a strict state progression with guards:

```
PENDING → ACTIVE → COMPLETED
   ↓         ↓
CANCELLED  CANCELLED
```

- `PENDING → ACTIVE` only allowed if an agent is assigned (`agentId !== null`)
- `COMPLETED` and `CANCELLED` allow no further transitions and are final
- Cancellation requires typing "CONFIRM" as a safety guard, with an optional reason

Role-Based Views

- **Supervisor**: sees all requests, can reassign agents, start/cancel requests, and sees the per-agent summary bar
- **Agent**: sees only requests assigned to them (`currentAgentId = 'a2'`), can start and complete their own work
- Role switching is available via both a dropdown and clicking the view title

Derived State Over Unnecessary State

Everything that can be calculated from existing state is derived during render rather than stored. Counts, summaries (per-agent stats), visible cards, and agent names are all computed on render.

Known Gaps & Shortcuts

- **Agent ID is hardcoded** — `currentAgentId = 'a2'` is hardcoded in `RequestList`. In a real app this would come from an auth context
- **No persistence on refresh** — all state resets on page refresh. Could add it to `localStorage` or a backend
- **No error handling** — since there's no real backend, there's no need for it, but a production version would need it

- **No pagination or virtualization** — the request list grows infintely as new requests are created. With more time, would add windowing (e.g. `react-window`)